SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year: 2025

Name: V. Poojya Lakshmi
Reg No: 192324273

Assignment 1 - Answers

In this assignment I have solved three questions: designing a finite state automaton for dates, writing regular expressions for the seven languages given, and finding the minimum edit distance between the words drive, brief and divers. I tried to explain the reasoning for every step so that the answers can be followed without needing the question paper next to it.

Question 1. FSA for absolute date expressions

What the question asks

The automaton has to accept date expressions such as *March 15*, *the 22nd of November* and *Christmas*. Only absolute dates are needed. Words like *today*, *tomorrow* or *the day before yesterday* depend on the day on which the sentence is spoken (they are deictic), so they are not part of the language. Each edge of the automaton must carry a word or a set of words, and I am allowed to use short names for sets of words so that the diagram does not become a mess of arcs.

Before drawing anything I listed the kinds of date phrases people actually write, because the automaton should cover all of them and not just the three examples:

- month followed by day: *March 15*, *March 15th*, *March fifteenth*
- day followed by month, with or without *the* and *of*: *15 March*, *the 22nd of November*, *the twenty-second of November*
- an optional year at the end: *March 15, 1999*, *15 March 1999*, *March 1999*
- an optional weekday at the start: *Monday, March 15*, *Friday the 13th of May*
- named holidays: *Christmas*, *New Year's Day*, *Christmas Eve*, *Independence Day*

Classes of words (shorthand used on the edges)

To keep the graph readable I grouped the words into classes and wrote only the class name on the edge. Any word of the class can be read on that edge. Words written in lower case on an edge (like *the* and *of*) are used as they are.

Class name	Words in the class
MONTH	January, February, March, April, May, June, July, August, September, October, November, December (and the short forms Jan, Feb, Mar, Apr, Jun, Jul, Aug, Sep, Sept, Oct, Nov, Dec)
WEEKDAY	Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday
DAYNUM	the plain numbers 1, 2, 3, ... , 31
ORD	the numbers with an ending: 1st, 2nd, 3rd, 4th, ... , 22nd, ... , 31st
ORDW	ordinals written as words: first, second, third, ... , twenty-second, ... , thirty-first
YEAR	four digit years such as 1947, 1999, 2025
HOLIDAY	Christmas, Easter, Halloween, Thanksgiving, New Year's, Valentine's, Independence, Republic, Diwali, Pongal
Eve | Day	just the two words Eve and Day (used after a holiday name)

I kept DAYNUM (plain number) separate from ORD (number with ending) on purpose. English allows *the 22nd of November* and *March 15*, but it does not allow *the 22 of November* or *15 of March* in normal writing. If both were in one class the machine would accept these wrong forms.

How the machine is organised

State q0 is the start state. The weekday is optional, so from q0 there is one edge for a weekday going to q1, and an empty (ε) edge that skips straight to q2. After the weekday there may be a comma, so q1 goes to q2 on a comma or on ε. State q2 is the real beginning of the date, and from it the machine can go in four directions depending on the next word:

- a MONTH word takes it to q3 (month first style)
- the word *the* takes it to q6, which then needs an ordinal (ORD or ORDW) and lands in q7
- an ordinal directly (*22nd*, *twenty-second*) takes it to q7, while a plain number takes it to q9
- a HOLIDAY word takes it to q12, which is already a final state

The complete automaton is drawn on the next page (Figure 1). The final states are drawn with a double circle: q4, q10, q11, q12 and q13.

Figure 1: FSA for absolute date expressions

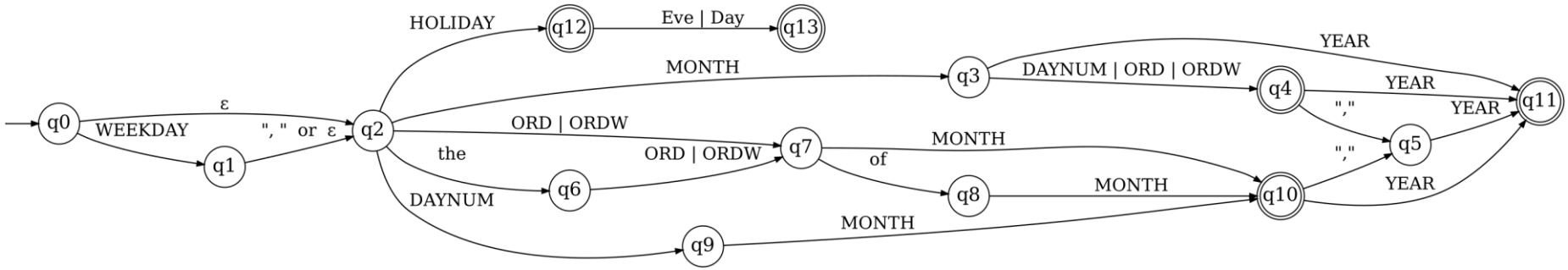


Figure 1. Complete automaton (double circles are accepting states, ε is an empty move)

Reading the figure: the label ORD | ORDW means that either an ORD word or an ORDW word can be read on that edge. The label ", " means a comma. The edge q0 to q2 labelled ε lets the weekday be skipped, and the edge from q1 to q2 can be taken with a comma or with nothing.

The states and what each one means

It helps to know what the machine has already seen when it is sitting in each state. This is the table I used while drawing the figure:

State	What has been read so far	Accepting?
q0	nothing yet (start state)	No
q1	a weekday, such as Monday	No
q2	optional weekday and comma finished, the date itself is about to begin	No
q3	a month name first, such as March	No
q4	month followed by a day, such as March 15 or March 15th	Yes
q5	a comma after a complete date, now waiting for a year	No
q6	the word the	No
q7	an ordinal day (22nd or twenty-second), with or without the	No
q8	the word of after an ordinal day	No
q9	a plain day number such as 15, now waiting for a month	No
q10	day and month in day-first order, such as the 22nd of November	Yes
q11	a complete date with the year, such as March 15, 1999	Yes
q12	a holiday name such as Christmas	Yes
q13	a holiday name with Eve or Day, such as Christmas Eve	Yes

Transition list

Written as a list, every edge of Figure 1 is given below (the same information as the picture, but easier to check line by line):

From	Input read	To	
q0	WEEKDAY	q1	
q0	ε	q2	skip weekday
q1	comma, or ε	q2	
q2	MONTH	q3	month first
q2	the	q6	
q2	ORD or ORDW	q7	
q2	DAYNUM	q9	
q2	HOLIDAY	q12	holiday
q3	DAYNUM, ORD or ORDW	q4	

From	Input read	To	
q3	YEAR	q11	March 1999
q4	comma	q5	
q4	YEAR	q11	
q5	YEAR	q11	
q6	ORD or ORDW	q7	
q7	of	q8	
q7	MONTH	q10	22nd March
q8	MONTH	q10	
q9	MONTH	q10	15 March
q10	comma	q5	
q10	YEAR	q11	
q12	Eve or Day	q13	

Strictly speaking, an FSA drawn with ε edges is a non-deterministic one (NFSA). That is fine here, because every NFSA can be converted to a deterministic one. In this case it is also very easy to remove the ε edges by hand: q0 can simply be given all the outgoing edges of q2 as well, and q1 gets the outgoing edges of q2 on top of its comma edge. I left the ε version in the figure because it is shorter to read.

The same machine in two smaller pieces

Figure 1 is wide, so here are the two main halves redrawn separately and bigger. Figure 2 is the month-first part (everything that starts from q3) and Figure 3 is the day-first part (everything that goes through q6 to q9 to q10).

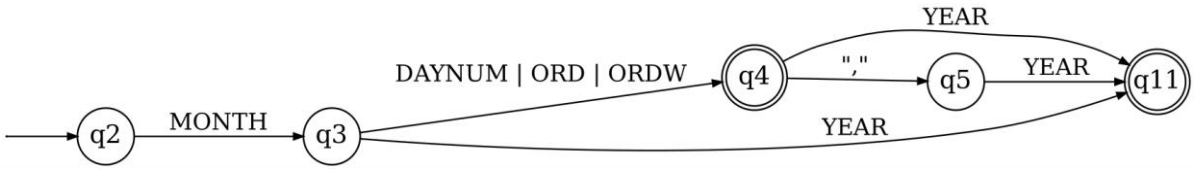


Figure 2. Month-first dates: March 15, March 15th, March 15, 1999, March 1999

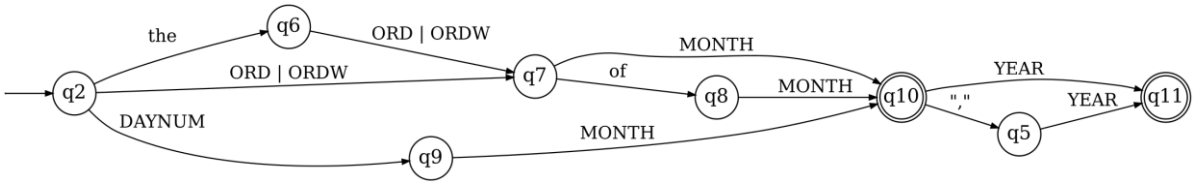


Figure 3. Day-first dates: the 22nd of November, 15 March, the twenty-second of November 1999

Both halves finish in q11 when a year is present. A year is optional in both of them, which is why q4 and q10 are accepting states by themselves.

Checking the automaton on examples

To be sure the machine does what I want, I traced some strings through it. The path shows the states visited, and the ε move from q0 to q2 is not written when there is no weekday.

Input string	Path through the states	Result
March 15	q0, q2, q3, q4	Accepted
the 22nd of November	q0, q2, q6, q7, q8, q10	Accepted
Christmas	q0, q2, q12	Accepted
Christmas Eve	q0, q2, q12, q13	Accepted
Monday, March 15th, 1999	q0, q1, q2, q3, q4, q5, q11	Accepted
15 March 1999	q0, q2, q9, q10, q11	Accepted
the twenty-second of November	q0, q2, q6, q7, q8, q10	Accepted
Friday the 13th of May	q0, q1, q2, q6, q7, q8, q10	Accepted
March the 15	q0, q2, q3, stuck on *the*	Rejected
the 22 of November	q0, q2, q6, stuck on *22* (not ORD)	Rejected
15 of March	q0, q2, q9, stuck on *of*	Rejected
Christmas Eve Day	q0, q2, q12, q13, stuck on *Day*	Rejected
tomorrow	q0, stuck on *tomorrow*	Rejected (deictic)

Limitations

The automaton is not perfect and I want to point out where it is too generous or too strict:

- It accepts impossible dates such as *February 31* or *April 31*, because DAYNUM is a single class for 1 to 31. This could be fixed by splitting MONTH into three classes (31-day months, 30-day months and February) and splitting DAYNUM to match, but the diagram would then have almost three copies of the month-first part.
- The class HOLIDAY lets through a few odd combinations like *Diwali Eve* or the bare word *Independence*. A cleaner way is to keep the full holiday names as one unit, but that needs a longer list of words.
- Moving holidays such as Easter or Thanksgiving are absolute names, but the actual calendar date changes every year, so the automaton only recognises the words and does not know which day they mean.

- Forms like *3/15/1999* or *15-03-1999* are not included. They could be added with another branch from q2 that reads digits and a slash or hyphen.
- Ordinals such as *the 22nd of November* with a missing *of* (*the 22nd November*) are accepted, which is common in British English, so I kept that.

Question 2. Regular expressions

I am using the *Perl notation* for all the answers (character classes in square brackets, `+`, `*`, `?`, `\b` for a word boundary, `\s` for white space, `^` and `$` for the start and end of the line, and `\1` to refer back to a group). For parts where it is possible I also give the minimal algebraic form of Section 2.3 (only concatenation, union written with `|`, and the Kleene star), and I say clearly when that is not possible.

A word means a string of letters separated from other words by white space or punctuation. Because of that I use `\b` (word boundary) in the patterns wherever a whole word has to be matched, so that a word inside a longer word is not accepted by mistake. The test tables below were checked by actually running the patterns.

(a) The set of all alphabetic strings

```
/^[A-Za-z]+$/
```

The class `[A-Za-z]` is any one letter, either lower or upper case, and the `+` says one or more of them. `^` and `$` make sure that nothing else is in the string. I used `+` and not `*` because the empty string has no letters and so I do not count it as an alphabetic string. When the strings are words inside a sentence, the version without anchors is used: `/\b[A-Za-z]+\b/`.

In the algebraic notation the same language is `[A-Za-z][A-Za-z]*`, which means one letter followed by any number of letters.

String	Matches?	Reason
hello	Yes	all letters
Humbert	Yes	capital and small letters both allowed
NLP	Yes	all capital letters
abc1	No	contains a digit
hi there	No	contains a space, so it is two words
don't	No	apostrophe is not a letter
(empty)	No	needs at least one letter

(b) All lowercase alphabetic strings ending in b

```
/^[a-z]*b$/
```

Here every character has to be a lowercase letter, so the class is only `[a-z]`. The star allows any number of lowercase letters at the start, and then the string must finish with the letter b. Since the star allows zero letters, the single letter `b` is also accepted, which I think is correct

because it is a lowercase string ending in b. Note that a capital B at the end does not count and neither does a digit before the b. For words inside text I would write `/\b[a-z]*b\b/`.

Algebraic form: `[a-z]*b`

String	Matches?	Reason
web	Yes	lowercase, ends in b
crab	Yes	lowercase, ends in b
b	Yes	zero letters and then b
bb	Yes	ends in b
bat	No	ends in t
Crab	No	capital C
cab1	No	ends in a digit
a	No	does not end in b

(c) Strings with two consecutive repeated words

```
/\b([A-Za-z]+)\s+\1\b/i
```

This one needs a memory. The part in brackets `([A-Za-z]+)` catches a word and stores it in register number 1. Then `\s+` allows one or more white space characters, and `\1` asks for exactly the same text that was stored. The `\b` at both ends makes sure that whole words are compared. Without the final `\b` the pattern would accept *the theory* because *the* is repeated at the start of *theory*, and with it that is correctly rejected. The `i` flag at the end makes it case insensitive, so that *The the* is also found.

This cannot be written in the minimal algebraic notation. Matching *any word followed by the same word* needs the machine to remember the word, and plain regular expressions cannot do that. The back reference `\1` is an extra feature of Perl style expressions.

String	Matches?	Reason
Humbert Humbert	Yes	same word twice
the the	Yes	same word twice
said that that is	Yes	that that
The the	Yes	case ignored because of the i flag
the bug	No	different words
the big bug	No	bug is not right after the other bug
the theory	No	theory is a different word from the
bug bugs	No	bugs is not the same as bug

(d) Strings over {a, b} where every a has a b before it and a b after it

```
/^(b(b|ab)*)?$/
```

I thought about this one by looking at what a valid string looks like. It cannot start with a, because that a would have nothing before it. It cannot end with a for the same reason on the other side. Two a's cannot be next to each other either. So a valid non-empty string starts with b and after that is made of the pieces `b` and `ab` only. Every piece ends in b, so each a that appears is automatically preceded by a b (the last letter of the piece before it) and followed by a b (the last letter of its own piece). For example `babab` is b + ab + ab. The outer brackets with `?` allow the empty string, which has no a at all and so satisfies the condition trivially. If the empty string should not be counted, the outer `(...)?` can be removed.

Algebraic form: `b(b|ab)*` together with the empty string (so `(b(b|ab)*)|ε`).

The language is also easy to show as a small deterministic automaton (Figure 4). S0 is the start state, S1 means the last letter was b (and the string so far is fine) and S2 means the last letter was an a, so a b is compulsory next. The extra state called dead catches every mistake.

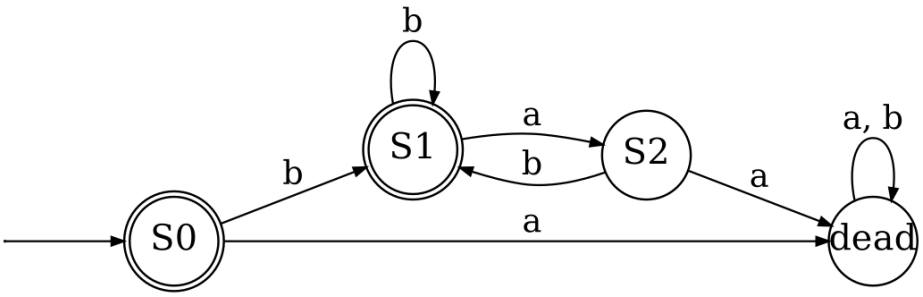


Figure 4. DFA for part (d). S0 and S1 are accepting.

String	Matches?	Reason
(empty)	Yes	no a in it
bbb	Yes	no a in it
bab	Yes	a has b on both sides
babab	Yes	both a's have b on both sides (the middle b is shared)
bbabb	Yes	ok
a	No	a has no b around it
ab	No	a at the start has no b before it
ba	No	a at the end has no b after it
baab	No	the two a's are next to each other
babba	No	last a has no b after it

(e) Starts with an integer at the beginning of the line and ends with a word at the end of the line

```
/^[0-9]+\b.*\b[A-Za-z]+[.,;:!?]*$/
```

The pattern is read from left to right. `^[0-9]+` means that the line begins with one or more digits, which is the integer. The `\b` after it makes sure that the number ends there, so something like *12abc* is not taken as an integer followed by text. Then `.*` allows anything in the middle, and `\b[A-Za-z]+` is the last word, which must begin at a word boundary so that it is a complete word. Finally `$` fixes it to the end of the line.

The question also says to deal with punctuation. A sentence normally ends with a full stop or similar mark, and I still want to call the last item a word. That is why I added `[.,;:!?]*` before the `$`, which allows any marks after the last word. If the end of the line should be a word with nothing after it, the simpler version `/^[0-9]+\b.*\b[A-Za-z]+$/` can be used.

Line	Matches?	Reason
12 apples are red	Yes	starts with 12, ends with red
7 days in a week.	Yes	full stop is allowed after week
1999 was a good year	Yes	integer 1999 at the start
5 cats, 6 dogs	Yes	ends with dogs
42	No	no word at the end
3 4	No	ends with a number, not a word
hello 5 world	No	does not start with an integer
12abc defg	No	12abc is not an integer

(f) Strings that contain both the word grotto and the word raven

```
/^(?=.*\bgrotto\b)(?=.*\braven\b).*$/i
```

The two words can come in either order, so a pattern like `grotto.*raven` alone is not enough. One way is to write both orders joined by a union:

```
/\bgrotto\b.*\braven\b|\braven\b.*\bgrotto\b/i
```

The first pattern uses two look-aheads. The part `(?=.*\bgrotto\b)` only checks that somewhere ahead on the line there is the whole word grotto, without using up any characters. After that the second look-ahead checks for raven starting again from the same place, and so the order does not matter. The second pattern, with the union sign, does the same job with plain Perl features and is easier to understand but longer.

The word boundary `\b` is very important in this question. The question says that words like *grottos* that only contain the word must not count. With `\b` on both sides, *grottos* fails because there is no boundary between the o and the s. The `i` flag lets *Raven* and *Grotto* with capital letters count as well. One more point is that *raven's* would be accepted, as the apostrophe also creates a boundary, and I think that is right because raven is still a separate word there.

String	Matches?	Reason
the raven sat in the grotto	Yes	both words present
grotto and raven	Yes	other order
The Raven flew into the Grotto.	Yes	capital letters and a full stop are fine
raven, grotto	Yes	comma is not part of the words
a raven	No	grotto is missing
grottos and ravens	No	only longer words
the grottoes had a raven	No	grottoes is not grotto

(g) Place the first word of an English sentence in a register

```
/^[\s"'‟‘(\[]*([A-Za-z]+(?:['’][A-Za-z]+)?)/
```

A register here means a stored group in brackets that can be used again later (like `\1` in the substitutions of the ELIZA example). The word to be saved is inside the round brackets: `([A-Za-z]+(?:['’][A-Za-z]+)?)`. It is one or more letters, and an optional apostrophe followed by more letters is allowed so that contractions like *Don't* and *It's* stay as one word. The `(?: )` is a group that does not make a register, so the first word is stored in register 1 only.

Punctuation is dealt with in the part before the bracket. `[\s"'‟‘(\[]*` skips any white space, quotation marks or opening brackets at the beginning, so that a sentence like `"Hello," she said.` stores *Hello* and not the quotation mark. The comma after Hello is outside the letters, so it is not stored. The word at the end of the sentence, as in *Run!*, is also stored without the exclamation mark.

To get the first word of every sentence in a longer text, and not just the first one, I would change the start of the pattern so that a sentence can begin either at the start of the text or after a full stop, question mark or exclamation mark followed by a space:

```
/(?:^|[.!?]\s+)["'‟‘(\[]*([A-Za-z]+(?:['’][A-Za-z]+)?)/g
```

If this pattern is used as a substitution, the stored word can be printed or reused with `\1`, for example `s/^[\s"'‘“(\[]*([A-Za-z]+).*/\1/` replaces the whole line with only its first word. A weak point is that abbreviations such as *Dr.* or *e.g.* also end with a full stop and could confuse the multi-sentence version.

Sentence	Register 1	Note
The cat sat down.	The	
"Hello," she said.	Hello	quotation mark skipped
(Maybe) it will rain.	Maybe	bracket skipped
Don't go there.	Don't	apostrophe kept
It's late.	It's	apostrophe kept
Run!	Run	exclamation mark not stored

Summary of Question 2

Part	Perl pattern	Algebraic form
a	/^[A-Za-z]+$/	[A-Za-z][A-Za-z]*
b	/^[a-z]*b$/	[a-z]*b
c	/\b([A-Za-z]+)\s+\1\b/i	not possible (needs memory)
d	/^(b(b|ab)*)?$/	b(b|ab)* | ε
e	/^[0-9]+\b.*\b[A-Za-z]+[.,;:!?]*$/	uses . and \b, Perl only
f	/^(?=.*\bgrotto\b)(?=.*\braven\b).*$/i	grotto.*raven | raven.*grotto
g	/^[\s"“‘(\[]*([A-Za-z]+(?:['’][A-Za-z]+)?)/	uses a register, Perl only

Question 3. Minimum edit distance: drive, brief and divers

Method

The minimum edit distance between two strings is the smallest total cost of operations (insertion, deletion, substitution) needed to change one string into the other. The question lets me choose the version of the distance. I will first use the Levenshtein distance in the form given in the textbook, where insertion costs 1, deletion costs 1 and substitution costs 2 (a substitution is the same as one deletion plus one insertion, so it costs 2). Then, at the end, I will repeat the work with substitution costing 1 to see whether the answer depends on that choice.

The source string is *drive* (length 5) in all cases, and the targets are *brief* (length 5) and *divers* (length 6). The table D has a row for each prefix of the source and a column for each prefix of the target. D(i, j) is the distance between the first i letters of drive and the first j letters of the target. The rule for filling it is:

```
D(i,0) = i          D(0,j) = j D(i,j) = min {  D(i-1,j) + 1
(delete source[i])                 D(i,j-1) + 1
(insert target[j])                 D(i-1,j-1) + c        (c =
0 if letters equal, else 2)  }
```

The first row and column are just the cost of building a string from nothing by insertions, or deleting everything. As an example of one cell, take D(1,1) for *d* against *b*. Delete d and have b to insert gives D(0,1) + 1 = 2. Insert b gives D(1,0) + 1 = 2. Substituting d by b gives D(0,0) + 2 = 2. The minimum is 2. For a cell where the letters match, such as D(2,2) for *dr* against *br*, the diagonal move costs nothing, so D(2,2) = D(1,1) + 0 = 2.

drive to brief

The rows are the letters of drive (d, r, i, v, e) and the columns are the letters of brief (b, r, i, e, f). The symbol # stands for the empty string. The shaded cells are the path found by the backtrace, explained below.

	#	b	r	i	e	f
#	**0**	1	2	3	4	5
d	1	**2**	3	4	5	6
r	2	3	**2**	3	4	5
i	3	4	3	**2**	3	4
v	4	5	4	**3**	4	5
e	5	6	5	4	**3**	**4**

Table 1. Distance matrix for drive and brief (substitution cost 2)

The bottom right cell is D(5,5) = 4, so the minimum edit distance between drive and brief is *4*.

Backtrace

To find which operations give this cost I start from the bottom right cell and go back to the cell that produced its value:

- D(5,5) = 4 comes from D(5,4) = 3 plus 1, which is an insertion of f.
- D(5,4) = 3 comes from D(4,3) = 3 by a diagonal move, since e = e (cost 0).
- D(4,3) = 3 comes from D(3,3) = 2 plus 1, a deletion of v.
- D(3,3) = 2 comes from D(2,2) = 2 by a diagonal move, i = i (cost 0).
- D(2,2) = 2 comes from D(1,1) = 2 by a diagonal move, r = r (cost 0).
- D(1,1) = 2 comes from D(0,0) = 0 plus 2, a substitution of d by b.

Writing the alignment out with the operations below it (s = substitute, d = delete, i = insert, a dot means the letters match):

drive	d	r	i	v	e	*
brief	b	r	i	*	e	f
operation	s (2)	. (0)	. (0)	d (1)	. (0)	i (1)

Total cost = 2 + 0 + 0 + 1 + 0 + 1 = 4. So drive can be changed into brief by replacing d with b, deleting v and inserting f at the end.

drive to divers

Now the target is divers (d, i, v, e, r, s), with the same rules. The table has one more column because divers has six letters.

	#	**d**	**i**	**v**	**e**	**r**	**s**
#	**0**	1	2	3	4	5	6
d	1	**0**	1	2	3	4	5
r	2	**1**	2	3	4	3	4
i	3	2	**1**	2	3	4	5
v	4	3	2	**1**	2	3	4
e	5	4	3	2	**1**	**2**	**3**

Table 2. Distance matrix for drive and divers (substitution cost 2)

Here D(5,6) = 3, so the minimum edit distance between drive and divers is *3*.

Backtrace

- D(5,6) = 3 comes from D(5,5) = 2 plus 1, an insertion of s.

- D(5,5) = 2 comes from D(5,4) = 1 plus 1, an insertion of r.
- D(5,4) = 1 comes from D(4,3) = 1 by a match, e = e.
- D(4,3) = 1 comes from D(3,2) = 1, a match, v = v.
- D(3,2) = 1 comes from D(2,1) = 1, a match, i = i.
- D(2,1) = 1 comes from D(1,1) = 0 plus 1, a deletion of r.
- D(1,1) = 0 comes from D(0,0), a match, d = d.

drive	d	r	i	v	e	*	*
divers	d	*	i	v	e	r	s
operation	. (0)	d (1)	. (0)	. (0)	. (0)	i (1)	i (1)

Total cost = 0 + 1 + 0 + 0 + 0 + 1 + 1 = 3. The letters d, i, v, e of drive are all kept in the same order inside divers. Only the r has to be removed, and r and s added at the end. That is why this alignment is cheap.

Which word is closer?

Pair	Distance (substitution = 2)	Operations
drive and brief	4	1 substitution, 1 deletion, 1 insertion
drive and divers	3	1 deletion, 2 insertions

With the standard costs, *drive is closer to divers*, with an edit distance of 3, compared with 4 for brief. Even though drive and brief look similar because they both have an r and an i in the middle and the same length, one of the changes needed is a substitution, which is expensive here. On the other side, divers keeps four of the five letters of drive in order.

Checking with substitution cost 1

Since I am free to choose the version of the distance, I also filled both tables again where a substitution costs 1 instead of 2 (this is the simpler Levenshtein distance used in many books). Only the diagonal cost changes, from 2 to 1 when the letters differ.

drive to brief, final distance 3:

	#	b	r	i	e	f
#	0	1	2	3	4	5
d	1	1	2	3	4	5
r	2	2	1	2	3	4
i	3	3	2	1	2	3
v	4	4	3	2	2	3

e	5	5	4	3	2	3

Table 3. drive and brief with substitution cost 1

drive to divers, final distance 3:

	#	d	i	v	e	r	s
#	0	1	2	3	4	5	6
d	1	0	1	2	3	4	5
r	2	1	1	2	3	3	4
i	3	2	1	2	3	4	4
v	4	3	2	1	2	3	4
e	5	4	3	2	1	2	3

Table 4. drive and divers with substitution cost 1

With unit cost substitution the distance to brief becomes 3, since d can be replaced by b, v deleted and f inserted, so only one operation is saved by making the substitution cheaper. The distance to divers stays 3, because its best alignment did not use any substitution. Therefore in this version the two words are at the same distance from drive, and a tie happens. This shows that the answer depends a little on the cost chosen for substitution.

Conclusion

Using the standard Levenshtein costs (insert 1, delete 1, substitute 2), the minimum edit distance between drive and brief is 4 and between drive and divers is 3, so *drive is closer to divers* and the edit distance is *3*. If substitution is given cost 1, both distances are 3 and the two words are equally close.